



Compiladores

Aula 6: Gramáticas Livres de Contexto

Prof. Alex Torquato Souza Carneiro
alex.carneiro@impacta.edu.br

Agenda

- Elementos das GLCs
- Árvores de derivação
- Exercícios

Elementos das GLCs

- Uma gramática livre de contexto é uma quádrupla $G = (N, T, P, S)$.
- N | Não-terminais
 - Variáveis sintáticas: **expr, stmt, decl**
- T | Terminais
 - Tokens que aparecem na entrada: **IF, ID, INT, ;**
- P | Produções
 - Regras de reescrita: **stmt $\rightarrow$ if (expr) stmt**
- S | Inicial
 - Não-terminal que representa o programa completo: **program**

Elementos das GLCs

- Uma gramática é “**Livre de contexto**” quando um não-terminal pode ser substituído sem depender dos símbolos ao redor.
- Outro tipo de gramáticas são as GSCs, “**Gramáticas Sensíveis ao Contexto**”, onde as regras de produção assumem que não-terminais podem ser substituídos de modo a depender dos símbolos ao redor.

Elementos das GLCs

- Através das gramáticas, o parser reconhece relações e hierarquias a partir das unidades reconhecidas pelo scanner.
- A gramática descreve construções como expressões, comandos, declarações e funções, exatamente a estrutura que a AST precisa preservar.

Elementos das GLCs

- Exemplo:

Entrada

caracteres

if (x + 1) y = 2 ;



Scanner

tokens

IF (ID + INT)
ID = INT ;



Parser

estrutura

stmt
└ expr
└ assign

Elementos das GLCs

- Ao final, a gramática deixa de ser apenas uma lista de regras e passa a orientar o parser.
1. Especificar:
Definir terminais, não-terminais, produções e símbolo inicial.
 2. Derivar:
Gerar sentenças e acompanhar derivações à esquerda e à direita.
 3. Estruturar:
Construir árvores de derivação e conectar a ideia à AST.



Árvores de derivação

- Notação: BNF e EBNF
- A forma de escrever as produções influencia a clareza da especificação.

BNF — forma básica

```
<stmt> ::= <if-stmt> | <expr-stmt>
```

```
<if-stmt> ::= IF ( <expr> ) <stmt>  
            ELSE <stmt>
```

Alternativas são separadas por |.

EBNF — forma estendida

```
stmt = if-stmt | expr-stmt ;  
if-stmt = "if", "(", expr, ")",  
          stmt, ["else", stmt] ;
```

[] indica elemento opcional.

Para o projeto MINIC:

Declare explicitamente program como símbolo inicial e registre as convenções adotadas pela equipe.

Árvores de derivação

- A separação em níveis codifica precedência e associatividade.

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T \times F \mid T \div F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

Onde:

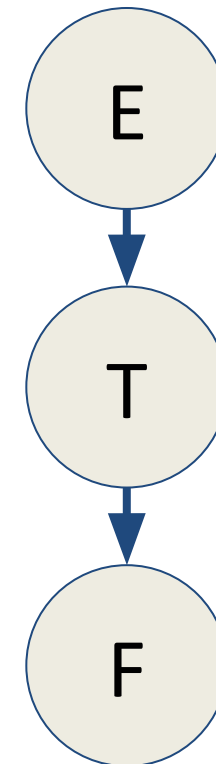
E expressão

T termo

F fator

Quanto mais abaixo, maior a precedência:

E soma/subtração $\rightarrow$ T multiplicação/divisão $\rightarrow$ F unidade



Árvores de derivação

- Uma derivação é uma sequência de aplicações de produções a partir do símbolo inicial.

$$E \Rightarrow E + T \Rightarrow T + T \Rightarrow id + T \Rightarrow id + id$$



Forma sentencial

Pode conter terminais e não-terminais.

Sentença

Contém apenas terminais.

Árvores de derivação

- A escolha do não-terminal a expandir antecipa estratégias de parsing:
 - derivação mais à esquerda ou
 - derivação mais à direita

Mais à esquerda

Sempre expande o não-terminal mais à esquerda.

$$\begin{aligned} E &\Rightarrow E + T \\ &\Rightarrow T + T \\ &\Rightarrow id + T \\ &\Rightarrow id + id \end{aligned}$$

Relacionada a abordagens top-down.

Mais à direita

Sempre expande o não-terminal mais à direita.

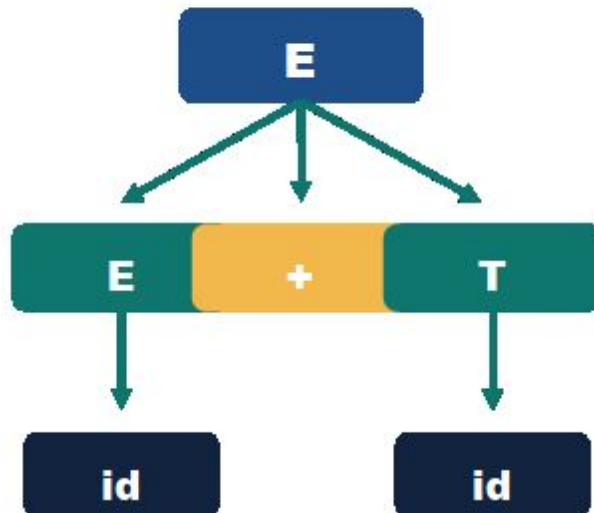
$$\begin{aligned} E &\Rightarrow E + T \\ &\Rightarrow E + id \\ &\Rightarrow T + id \\ &\Rightarrow id + id \end{aligned}$$

A análise bottom-up recupera a derivação mais à direita em ordem reversa.

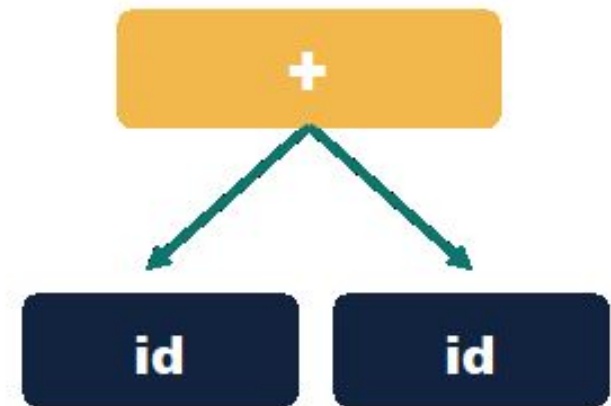
Árvores de derivação

- A árvore de derivação explica a gramática; a AST preserva a estrutura essencial.

Árvore de derivação



AST



Remove detalhes puramente sintáticos; preserve operadores e operandos.

Árvores de derivação

- Para o compilador MINIC, a gramática será a especificação que orienta o parser e a AST.

program $\rightarrow$ declaration-list

declaration-list $\rightarrow$ declaration declaration-list | ϵ

declaration $\rightarrow$ var-declaration | fun-declaration

var-declaration $\rightarrow$ type ID ;

stmt $\rightarrow$ expr-stmt | compound-stmt | if-stmt

expr-stmt $\rightarrow$ expr ; | ;

Exercícios

- Elabore as ASTs para as seguintes expressões:

$$x = 2 + 5 * 2;$$

$$x = 2 * 5 - 2;$$

$$x = 2 * 7 - 5 * 2;$$

$$x = 2 - 5 * 2 + 3;$$